Consumer–Resource Models on Spatial Networks
A Guide to the Code and Calculation Programs

Project documentation

26 October 2026

Abstract

This project implements a family of ecological consumer–resource models, with local population interactions and movement among connected patches. It combines deterministic population equations, individual stochastic simulations, probability-distribution dynamics, and statistical inference. This guide groups all fourteen directories in `MODEL_CALCULATIONS` by their scientific purpose and describes the shared implementation. Descriptions refer to the checked-in source; illustrative comments and proposed extensions are distinguished from active calculations. This is a source-based guide, not a report of successfully executed numerical experiments.

Contents

1 Scientific purpose and numerical approaches **1**

2 Shared implementation and program organization **2**

3 Library demonstrations and numerical examples **2**
3.1 EXAMPLE-MAIN-RANDOM . . . 2
3.2 EXAMPLE-MAIN-GILLESPIE . . . 3
3.3 EXAMPLE-MAIN-NUMERICAL-INTEGRATION . . . 3

4 Temporal dynamics and visualization **3**
4.1 TEMPORAL_EVOLUTION . . . 3
4.2 TEMPORAL_EVOLUTION_MASTER_EQUATION . . . 3

5 Fitting time series and inspecting fitted solutions **4**
5.1 TEMPORAL_EVOLUTION_ESTIMATION . . . 4
5.2 TEMPORAL_EVOLUTION_ESTIMATION_MULTIPLE . . . 4
5.3 TEMPORAL_EVOLUTION_ESTIMATION_SCAN . . . 4
5.4 TEMPORAL_EVOLUTION_SOL_INSPECTION . . . 4

6 Parameter landscapes and likelihood visualization **5**
6.1 VISUAL_2DIM_PARAMETER_SCAN_FUNCTION . . . 5
6.2 VISUAL_2DIM_PARAM_SCAN_LK_ERROR_MODEL . . . 5
6.3 VISUAL_2DIM_PARAM_SCAN_LK_STATIONARITY . . . 5
6.4 VISUAL_2DIM_PARAM_SCAN_LK_DYNAMIC . . . 5

7 LaTeX reporting **5**
7.1 LATEX_PARAMETER_TABLE_FILE_GENERATION . . . 5

8 Building, running, and choosing a workflow **6**

9 Scope and reproducibility notes 6

1 Scientific purpose and numerical approaches

Resources and consumers interact within patches connected by a network. Movement between patches is represented by random walks. Depending on the selected model, consumers may search, handle resources, or interfere with other consumers. The model family includes Holling type II feeding, multiple resource types, and interference mechanisms associated with Beddington–DeAngelis functional responses. Other model variants share the same parameter and spatial infrastructure, but support for each numerical method varies by model.

Three complementary descriptions are implemented. A deterministic calculation integrates population variables, schematically

$$\frac{d\boldsymbol{x}}{dt} = \boldsymbol{f}(\boldsymbol{x}; \boldsymbol{\theta}), \tag{1}$$

where the vector field includes local ecological interactions and spatial exchange. A stochastic calculation instead follows discrete population-changing events. In a standard Gillespie step, the total propensity is $a_0 = \sum_j a_j$, the waiting time is exponentially distributed with rate a_0, and event j is selected with probability a_j/a_0. Optimized variants reduce the work required to select events and update affected rates.

A master-equation calculation evolves probabilities over population configurations rather than a single population trajectory:

$$\frac{dp(\boldsymbol{n}, t)}{dt} = \sum_j \left[a_j(\boldsymbol{n} - \boldsymbol{\nu}_j) p(\boldsymbol{n} - \boldsymbol{\nu}_j, t) - a_j(\boldsymbol{n}) p(\boldsymbol{n}, t) \right]. \tag{2}$$

These equations explain the roles of the numerical methods; individual model files supply their actual rates, state variables, and assumptions. Finite configuration spaces can become much larger than the corresponding population ODE system.

2 Shared implementation and program organization

Each calculation directory is an application or reporting workflow with its own makefile. Most full applications contain `main.c`, `global.h`, `extern.h`, `default.c`, command-line parsing, and local model drivers such as `MODEL.c` and `MODEL_STO.c`. They reuse source files outside their own directory. Several configuration fragments are included directly as C source, so reading only `main.c` does not reveal every default or operation.

Core structures. `Include/MODEL.h` assembles model-dependent definitions and shared structures. `Parameter_Table.c`, `Parameter_Model.c`, `Community.c`, and `Time_Control.c` manage parameters, patch populations, and observation times. `Observed_Data.c` and `Definition_Fitting_Structure` support inference.

Model-specific implementations. `Assign_Model_Variables` maps state variables to vector entries and labels. `Definition_Numerical_Integration` contains integration support and model equations; its `ODE_Definitions` subtree contains vector fields and Jacobians. `Definition_Stochastic_Realizations` implements initialization, propensities, event execution, and rate updates.

Other numerical components. `Definition_Master_Equation`, `Definition_Fixed_Points`, and `Definition_R0_Reproductive_Number` provide probability dynamics, equilibrium/stability calculations, and reproductive-number functions for supported models.

Parameters and data. `Definition_Parameter_Space` defines search ranges. `Definition_Error_Model` supplies observation-error support. `Definition_Time_Trend_Dependence` implements time-dependent forcing and trends.

Utility libraries. `Library/GSL_RANDOM` supplies random and sampling utilities; `Library/IO_Library` handles data and parameter files; `Library/Optimization_Library` contains minimization and likelihood functions; `Library/C_NumRecip` contains Numerical Recipes routines.

Output. Root-level assignment and output functions supply names, symbols, parameter tables, and derived observables. Plotting uses an external CPGPLOT library and its underlying graphics dependencies.

The compile-time `MODEL` variable selects a model implementation. Runtime `-y0` selects its numeric model code and must agree with the compiled choice. A directory therefore identifies the kind of analysis, while the model selection identifies the ecological system being analyzed. Not every directory/model combination is implemented.

3 Library demonstrations and numerical examples

These small programs are useful for understanding numerical building blocks. They are demonstrations with printed or plotted results, rather than an automated regression-test suite.

3.1 `EXAMPLE-MAIN-RANDOM`

Initializes a GSL random-number generator, reports its identity, prints uniform and Gaussian draws, and exercises `Discrete_Sampling` on a ten-entry array of event weights. The weights are not normalized probabilities. The program prints sampled event indices and releases the allocated storage. Its `main` takes no command-line arguments.

3.2 `EXAMPLE-MAIN-GILLESPIE`

Demonstrates direct stochastic simulation of a consumer–resource system using discrete population counts. The local `Update_Total_Rate` routine computes event rates from the current state; the main loop selects an event, advances time, and changes the relevant counts. Parameters and run controls are embedded in this example. Use it to inspect the mechanics of event-based simulation before following the larger spatial framework.

3.3 `EXAMPLE-MAIN-NUMERICAL-INTEGRATION`

Compares a single stochastic realization of the consumer–resource example described in its source as the Stollenberg model with numerical integration of the corresponding three-variable ODE system. It writes `Temporal_Evolution_Stochastic.dat` and `Temporal_Evolution_Deterministic.dat`. With graphics enabled, it plots the two descriptions for comparison. Its makefile names the executable `cotton`; unlike the two smaller examples, it parses command-line parameter overrides. See the opening comment in `main.c` for the local `-p` options.

4 Temporal dynamics and visualization

Visualization is a shared capability of several applications, rather than a separate kind of biological model. The programs below primarily calculate evolution in time; plots and files expose those results.

4.1 TEMPORAL_EVOLUTION

Runs deterministic and stochastic temporal dynamics for the selected supported model. The main program loads defaults and command-line overrides, constructs parameter and time structures, initializes random sampling, and dispatches to local deterministic and stochastic drivers. Depending on the model and compilation flags, equilibrium calculations and graphical output are also included.

Users choose spatial structure, initial conditions, ecological rates, output variables, time interval, and number of stochastic realizations. The plotted deterministic trajectory can be compared with stochastic results. Certain feeding-experiment models maintain a constant total number of consumers. The multiresource Holling II branch resets the attack/handling vectors and resource levels before calculation. Model-specific setup therefore matters even when the same command-line parameter names are used.

4.2 TEMPORAL_EVOLUTION_MASTER_EQUATION

Extends temporal analysis to the probability distribution of the stochastic process. The main program runs the deterministic driver, conditionally invokes a stochastic-realization driver, restores the parameter values, and invokes `M_O_D_E_L___M_E` for explicitly supported models. In the current main file these branches are `DIFFUSION_BD_2D`, `DIFFUSION_HII_1D`, and `DIFFUSION_HII_nD`.

The central distinction is between a distribution over configurations and an individual stochastic path. This program supports comparing probabilistic predictions with population dynamics for those models; it should not be assumed to provide a master equation for every model in the repository. The stochastic driver is guarded by a preprocessor symbol, so its inclusion must be checked against the local makefile.

5 Fitting time series and inspecting fitted solutions

The fitting applications connect observed outputs to model predictions through an objective function in the optimization library. They configure parameter bounds and may also configure initial-condition and observation-error parameters. A typical criterion is a negative log likelihood,

$$\widehat{\boldsymbol{\theta}} = \arg\min_{\boldsymbol{\theta}\in\Theta}[-\log L(\boldsymbol{\theta}; D)]. \tag{3}$$

The actual objective depends on the selected callback; ODE predictions with an observation-error model and likelihoods derived from jump-process distributions are distinct approaches in this project.

5.1 TEMPORAL_EVOLUTION_ESTIMATION

Fits an ODE model to a data matrix read from a file. The documented format places observation times in the first row and each observed variable in a subsequent row. Time-dependent parameters may require another input file. The current objective callback is `GSL_Function_to_Minimize_Error_Model`, and fitting is performed by `GSL_Minimization_Driver`.

Repeated attempts explore different starting guesses within the configured bounds. Accepted results are recorded in `Full_Parameter_Set.dat`, with model parameters, any fitted initial conditions, error parameters, and the negative log likelihood. Here the repetition control is used for fitting attempts; it should not automatically be interpreted as the number of stochastic paths. Local code also makes assumptions about the fitted system and initialization that must be checked before applying it to another model.

5.2 TEMPORAL_EVOLUTION_ESTIMATION_MULTIPLE

Applies the fitting workflow to multiple observed-data files. An outer loop selects the data file, while an inner loop performs repeated optimization attempts. It writes numbered ordered results such as `Full_Parameter_Set_Ordered_0.dat`. This supports comparing fitted parameters across data sets. The intermediate `Full_Parameter_Set.dat` is reopened for each data set, so it is not a combined permanent record of all fits.

5.3 TEMPORAL_EVOLUTION_ESTIMATION_SCAN

The source comments propose estimating relative parameter-recovery accuracy across stochastic realizations and eventually scanning that accuracy over parameter space. However, the proposed `Function_Relative_Accuracy` scan is shown as a commented design sketch. The active main program reads an observed-data file and performs repeated error-model minimizations, writing `Full_Parameter_Set.dat`. Treat this directory as a fitting workflow with a planned accuracy-scan extension, not as a completed generic recovery-accuracy scan.

5.4 TEMPORAL_EVOLUTION_SOL_INSPECTION

Reads previously obtained parameter sets and evaluates them against observed data. It sets `F->Minimization=0`, uploads each saved solution, and uses the fitting machinery in evaluation mode. This is the appropriate workflow for inspecting alternative fitted trajectories and objective values without launching a new optimization. The current loop asserts a single cell and a single resource, and initializes the total population from the first empirical value; these restrictions limit direct reuse for other systems.

6 Parameter landscapes and likelihood visualization

These applications evaluate a scalar function on a two-parameter grid. The parameter-space structure supplies ranges and grid resolution. With plotting enabled, the resulting arrays can be displayed as colored maps and, for the likelihood applications, profiles. A grid minimum is an estimate on the sampled grid; its precision depends on the chosen ranges and resolution.

6.1 VISUAL_2DIM_PARAMETER_SCAN_FUNCTION

Maps model-derived quantities that do not require an observed-data likelihood, such as coexistence or stability classifications. The generic interface accepts a function of `Parameter_Table`. Although comments discuss several possible functions, the active scan in `main.c` uses `Calculate_Type_of_Coexistence` and writes `Coexistence_Condition_Bis.dat`. Both axes are currently configured as linear. The directory also contains `main_R0.c`, an alternative source concerning reproductive-number calculations; its presence does not change the callback in the primary main program.

6.2 VISUAL_2DIM_PARAM_SCAN_LK_ERROR_MODEL

Scans the objective `GSL_Function_to_Minimize_Error_Model` over two model parameters. It reads empirical data or generates pseudo-data through a model driver, depending on the input configuration. It produces a negative-log-likelihood landscape, identifies a grid minimum, and provides graphical interpretation when enabled. Its statistical model combines model predictions with an observation-error formulation, unlike the jump-process likelihood programs below.

6.3 VISUAL_2DIM_PARAM_SCAN_LK_STATIONARITY

Analyzes likelihoods based on stationary distributions of supported stochastic feeding models. Model-dependent selection routines choose binomial, multinomial, or Beddington–DeAngelis likelihood callbacks, including a marginal likelihood when appropriate. The workflow supports simulated data, scans the selected parameters, identifies a grid minimum, and calculates likelihood profiles and confidence intervals. Its main file writes confidence-interval summaries. Stationarity here concerns the probability distribution, not just an equilibrium of the deterministic ODEs.

6.4 VISUAL_2DIM_PARAM_SCAN_LK_DYNAMIC

Analyzes time-dependent likelihoods of supported jump-process models. Such likelihoods use transient distributions, obtained analytically for some cases or through master-equation calculations. This contrasts with sampling from a stationary distribution. The program supports pseudo-data generation, repeated analyses, a two-parameter likelihood grid, grid-based estimates, profiles, and confidence-interval summaries. The main file prompts for a repetition count and writes files including `MLE_0_HII.dat`, `MLE_1_HII.dat`, and corresponding confidence-interval files. These filenames are embedded in the source and do not imply unrestricted model support. A model-selection routine explicitly limits the supported likelihoods.

7 LaTeX reporting

7.1 LATEX_PARAMETER_TABLE_FILE_GENERATION

This directory is a document-building workflow, not a C simulation executable. Its `Parameter_Table_Master.tex` includes `Latex_Parameter_Table.tex`; the makefile supplies LaTeX, PDF, bibliography, and PostScript targets. C applications elsewhere can produce parameter-table fragments using the root-level LaTeX-writing functions. This directory formats such a fragment into a standalone report. Existing auxiliary and log files are typesetting artifacts.

8 Building, running, and choosing a workflow

Build the desired application from its own calculation directory. For example, the documented temporal workflow is:

```
cd MODEL_CALCULATIONS/TEMPORAL_EVOLUTION
make MODEL=DIFFUSION_HII_nD
./DIFFUSION_HII_nD -h
```

This expresses the intended interface; it is not a verified successful build of the current checkout. There is no top-level makefile in this checkout, despite references to recursive builds in the README. Makefiles assume particular home-directory paths and link auxiliary static libraries that must be available first. The toolchain includes GCC, GNU Make, GFortran for some final link steps, GSL, and graphics dependencies. Some utility builds use R headers. Graphics depend on a separate CPGPLOT checkout, PGPLOT, and related plotting libraries.

Compilation controls include `MODEL`, `CPG`, `STATIONARITY`, `STO_REA`, `STO`, `BT`, and `RE`, where present in the local makefile. They select model source, graphical representation, stationary calculations, stochastic realizations, and event-selection optimizations. A typical non-graphical setting is `CPG=NON_CPGPLOT_REPRESENTATION`; some makefiles nevertheless retain graphics libraries in their link commands. Changing model or compilation flags requires rebuilding affected objects, since the makefiles do not reliably encode flag changes as dependencies.

Common runtime option families include `-HM`, `-HX`, and `-HY` for spatial dimensions; `-HS` for resource/species configuration; `-tn`, `-t0`, and `-t1` for observation times; `-n` and `-v0` onward

for output selection; `-H` options for ecological parameters; and `-G` options for graphics. Fitting examples additionally use parameter indices, bounds, data-file options, and error-model settings. These interfaces are application-specific: consult the local `argumentControl.c`, included parsing fragments, and examples in `main.c`. The meaning of a parameter can also change with the selected model.

A practical scientific workflow is to explore a supported model in `TEMPORAL_EVOLUTION`, fit a compatible observed time series with `TEMPORAL_EVOLUTION_ESTIMATION`, inspect retained solutions with `TEMPORAL_EVOLUTION_SOL_INSPECTION`, and examine parameter uncertainty with the appropriate likelihood scan. Use the master-equation program when the probability distribution itself is the object of study. Use the small examples to understand or check individual numerical components.

9 Scope and reproducibility notes

The source mixes reusable numerical components with experiment-specific choices. Some applications contain interactive key presses, hardcoded filenames, single-patch assumptions, or model-specific parameter resets. Run experiments in a controlled working directory and retain input data, parameter values, compilation options, and random-seed settings. Time-based random seeds lead to different realizations between runs.

The initial inspection found a concrete build obstacle: a dry run for `TEMPORAL_EVOLUTION` with `MODEL=DIFFUSION_HII_nD` requests a missing model-specific `Temporal_Dynamics_Aux_Functions.c`. Auxiliary archive libraries were also absent from the checkout, and some makefiles contain stale project paths. These are build observations, not reasons to infer that all scientific workflows are invalid. Availability must be checked for the particular application and model before numerical use.

For further detail, begin with the repository `README.md`, the selected application's `main.c` and makefile, then follow its model drivers and objective callbacks into the shared directories. The categories in this guide describe analysis tasks; visualization, file output, and parameter reporting cut across several categories.